PneumoScan

Aide au triage automatise des radiographies thoraciques

Anis Aissani

Universite de Rouen Normandie — L3 Informatique

2025 – 2026

Encadrant : Prof. C. Petitjean • Binome : Mohammed Elakef Zenagui

Plan

Contexte et problematique

Architecture technique

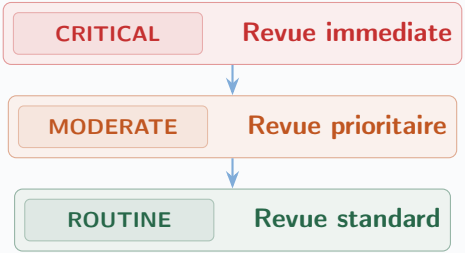
Demonstration de l'application

Demonstration en direct

Qualite et bilan

Contexte

- La pneumonie cause **2,5 millions de deces** par an dans le monde.
- Diagnostic repose sur la lecture de radiographies thoraciques.
- Files d'attente traitees en FIFO : un cas grave peut attendre trop longtemps.



Objectif

Developper un outil de triage automatise qui classe chaque radiographie en priorite clinique, tout en garantissant la tracabilite et la securite des acces.

Problematique et organisation du projet

Problematique

Comment integrer un modele ML pre-entraine dans une architecture web securisee garantissant tracabilite et rejet des entrees hors-distribution ?

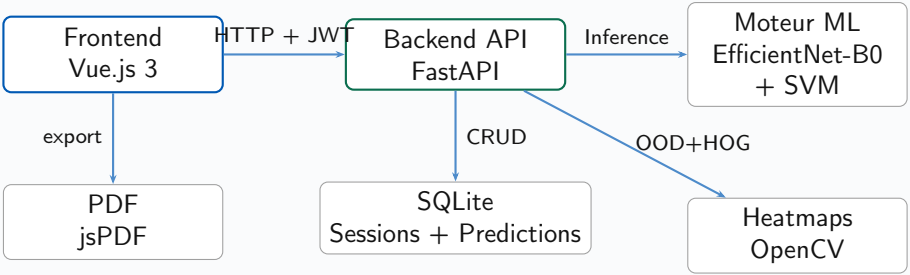
Repartition du travail

- **Binome** : entrainement et validation des modeles (HOG, LBP, Haralick, ResNet, EfficientNet)
- **Mon apport** : architecture logicielle complete — API, interface, securite, deploiement

Methodologie : UP adapte	
Inception	2 sem.
Elaboration	4 sem.
Construction	8 sem.
Transition	2 sem.

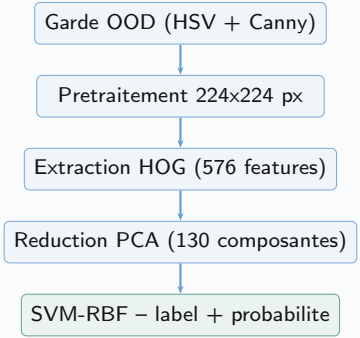
Prototype academique, non certifie dispositif medical

Vue d'ensemble



Pipeline d'analyse et architecture modulaire

Pipeline image



Modules backend

`main.py`	Routes FastAPI
`services.py`	Logique metier
`predictor.py`	Pipeline ML
`database.py`	SQLite + triage
`auth.py`	JWT / RBAC
`utils.py`	Validation

Modele retenu

EfficientNet-B0 + SVM

Accuracy 86.2% **F1 88.4%**

Fallback deploye : HOG + PCA + SVM (F1 81.4%)

Securite et tracabilite

Authentification JWT / RBAC

- 2 roles : `radiologist` et `admin`
- Token HS256, TTL 8h (1 shift hospitalier)
- Mots de passe haches bcrypt
- Stockage dans `sessionStorage` uniquement

Sessions anti-zombie

- A chaque reconnexion, les sessions ouvertes precedentes sont automatiquement cloturees
- Chaque prediction est tracee : operateur, horodatage, session

Regle de triage

$$\text{CRITICAL} = \hat{y} = P \ \wedge \ p > 0.85$$

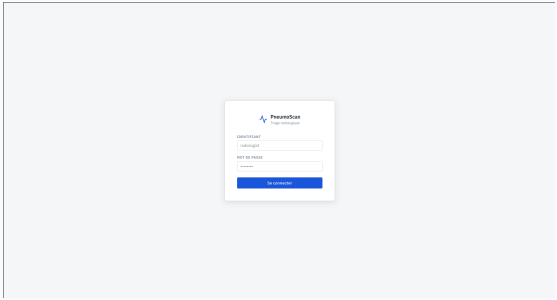
$$\text{MODERATE} = \hat{y} = P \ \wedge \ 0.15 \leq p \leq 0.85$$

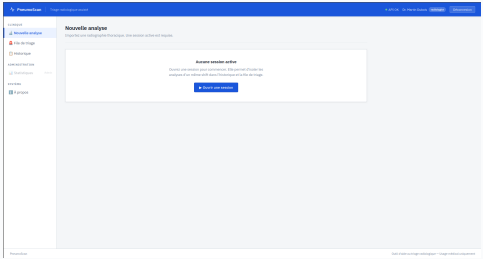
$$\text{ROUTINE} = \hat{y} = N \ \vee \ p < 0.15$$

Anonymisation RGPD

Masquage noir irréversible des coins (zones PACS contenant les donnees patient)

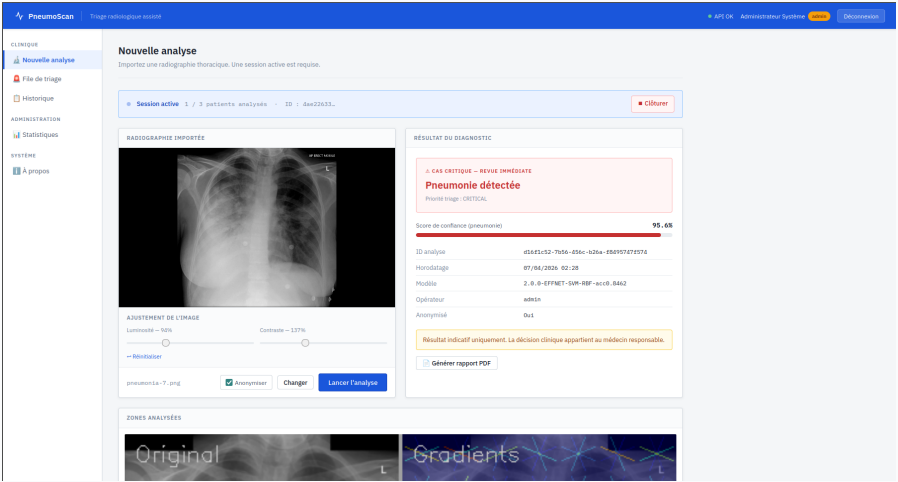
Connexion et ouverture de session



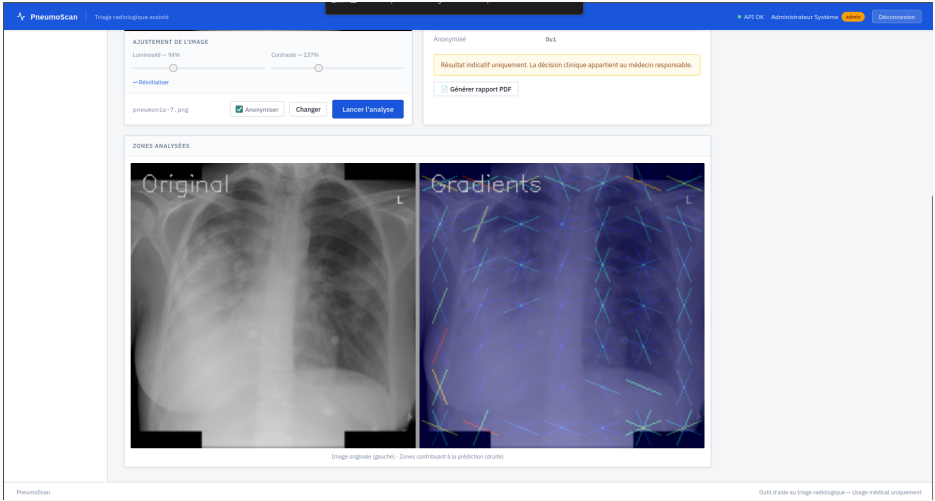


- Connexion securisee par JWT
- Sans session active, l'analyse est bloquee
- Chaque session rattache les analyses a un shift et un operateur

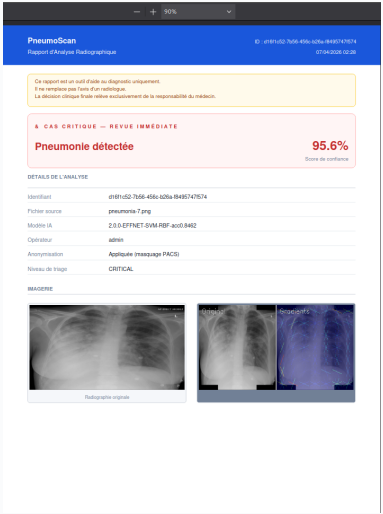
Analyse radiographique



Explicabilite visuelle : zones analysees



Rapport PDF genere automatiquement



90%

PneumoScan
Rapport d'Analyse Radiographique

ID : d181c52-7b56-456c-b26a-1849574767 4
07/04/2026 02:28

Ce rapport est un outil d'aide au diagnostic uniquement.
Il ne remplace pas l'avis d'un radiologue.
La décision clinique finale relève exclusivement de la responsabilité du médecin.

⚠ CAS CRITIQUE — REVUE IMMÉDIATE

Pneumonie détectée **95.6%**
Score de confiance

DÉTAILS DE L'ANALYSE

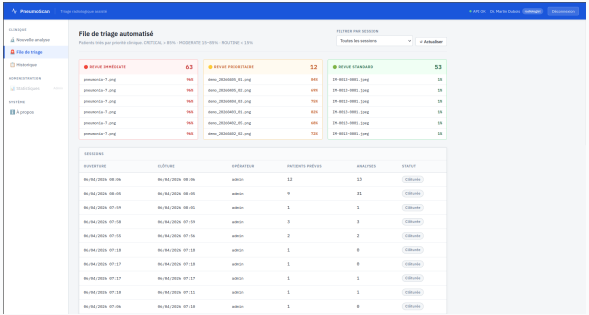
Identifiant	d181c52-7b56-456c-b26a-1849574767 4
Fichier source	pneumonia-7.png
Modèle IA	2.0.0-EFFNET-SVM-RBF-acc0.8462
Opérateur	admin
Anonymisation	Appliquée (masquage PACS)
Niveau de triage	CRITICAL

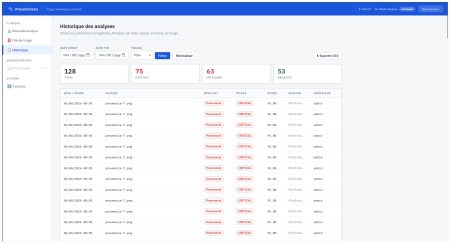
IMAGERIE

Radiographie originale

Original
Heatmap

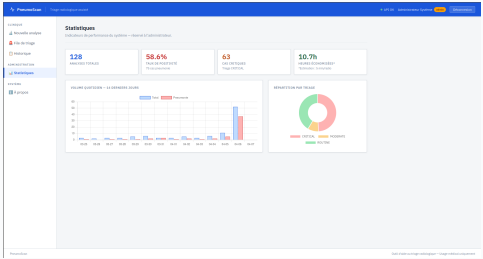
File de triage et historique

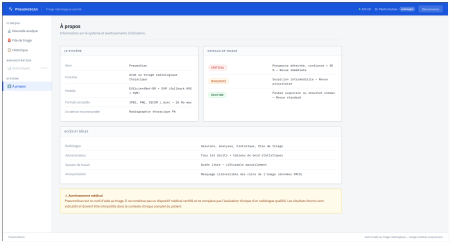




- 3 colonnes : CRITICAL / MODERATE / ROUTINE
- Historique filtrable + export CSV

Tableau de bord et page a propos





- Reserve au role `admin`
- Volume, positivite, repartition triage
- Estimation heures economisees

- Perimetre d'usage clairement affiche
- Avertissement medical integre

Demonstration en direct

Lancer :	`docker compose up`
Interface :	`http://localhost:3000`
API :	`http://localhost:8000/docs`
Radiologue :	`radiologist / radio123`
Admin :	`admin / admin123`

Parcours : connexion → session → analyse → triage → PDF → statistiques

Tests et qualite logicielle

Suite pytest — 32 tests, 32 passes

`TestValidation`	6 cas
`TestPreprocessing`	7 cas
`TestHOGFeatures`	3 cas
`TestTriageLogic`	9 cas
`TestAnonymization`	3 cas

Architecture SOLID

Separation claire des responsabilites (controller / service / data / ML)

Resultats du modele (binome)

Approche	Acc.	F1
HOG + SVM	74%	81.4%
HOG + LR	84%	88.4%
ResNet50 + SVM	80%	85.6%
EffNet + SVM	**86.2%**	**88.4%**
Concat + LR	100%	suspect

Modele retenu : EfficientNet-B0 + SVM

Bilan et perspectives

Ce qui a ete realise

- API REST FastAPI + interface Vue.js 3
- Securite JWT/RBAC + sessions anti-zombie
- Garde OOD + pipeline ML integre
- Triage 3 niveaux, historique, PDF, BI
- 32 tests unitaires passing
- Deploiement Docker Compose complet

Perspectives

- Support DICOM (fichiers hospitaliers reels)
- Fine-tuning EfficientNet-B0 sur donnees ciblees
- Explicabilite avancee (Grad-CAM, SHAP)
- Migration SQLite → PostgreSQL
- CI/CD et monitoring continu

En une phrase

PneumoScan demontre qu'un composant ML peut etre integre dans une architecture web robuste, securisee et deployable, avec une vraie valeur d'usage clinique.

Questions ?

Merci pour votre attention

GitHub : `github.com/Anis-Aissani/Pneumonia-Detection-App`

API docs : `http://localhost:8000/docs`